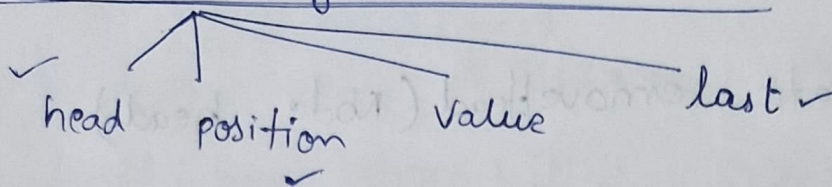
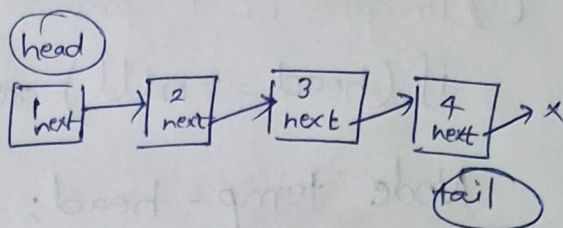


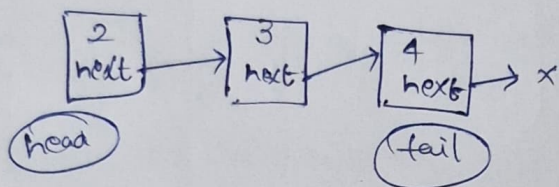
Deletion of Node in LL



i) Deletion of head



* just by moving the head to 2,

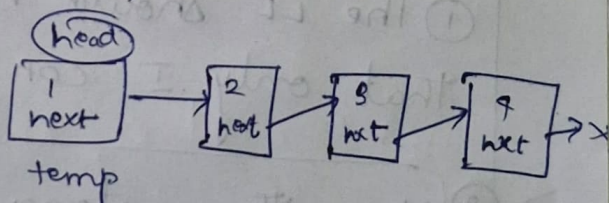


* But still the node is somewhere, But in C++ we use free() or delete.

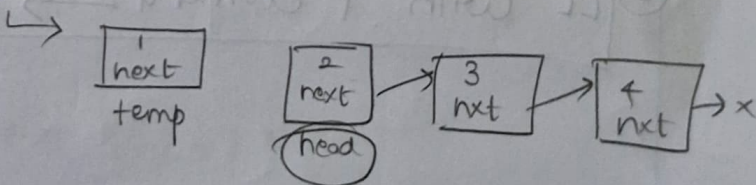
In Java we do not require those bcoz we have garbage collector

```
private static Node (Node head)
```

```
{  
  Node temp = head
```



```
  head = head.next
```



```
  return head;
```

```
}
```

Code with Edge Cases

```
private static removeHead (Node head)
```

```
{ // Imagine, LL is provide is empty
```

```
if (head == null) return head
```

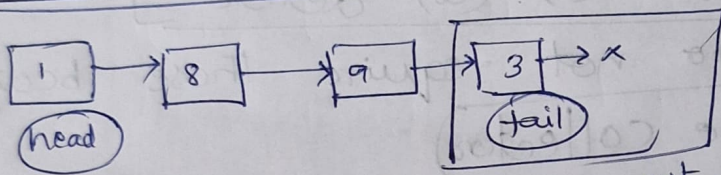
```
Node temp = head; X
```

```
head = head.next;
```

```
return head;
```

```
}
```

Deletion of Tail



stop before ^{last} element

① The LL should have atleast 2 element. So that only I can stop before Last

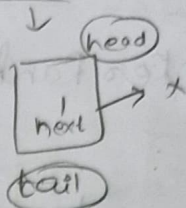
② LL with 1 element → delete that element

private static Node deleteNode (Node head)

{ Node temp = head
// Edge case if given LL is empty (or) LL $\rightarrow$ 1 element

$\rightarrow$ if (head == NULL || head.next == NULL)

{
return NULL;
}



// If LL $\rightarrow$ has 2 element or more than that

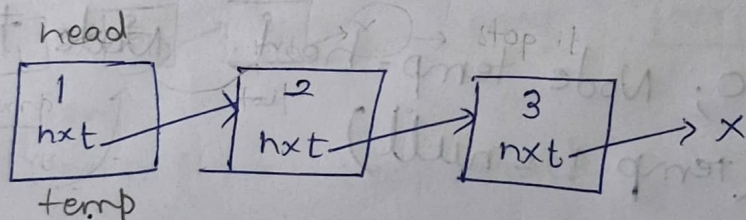
$\rightarrow$ ① while (temp.next.next != NULL)

{
temp = temp.next

}

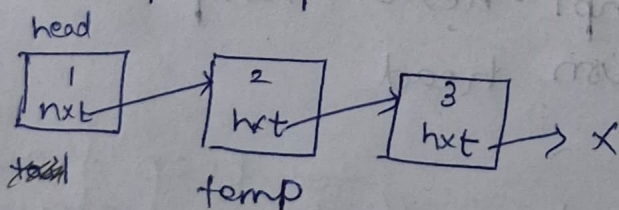
temp.next = null

return head



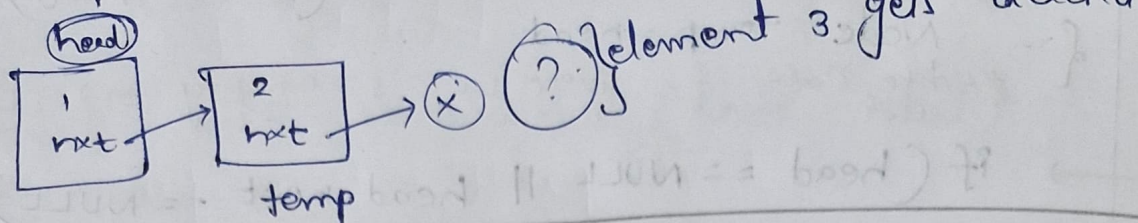
temp.next $\Rightarrow$ ② . next = 3 $\neq$ NULL $\checkmark$

temp = temp.next



temp.next $\Rightarrow$ ③ . next $\neq$ NULL $\times$

temp.next = NULL



// HackerRank delete Node at Nth position

Delete at the rth position

private static Node (Node head, int position)

{

// if the given LL is empty

if (head == null) return null;

// if ask to delete at position of head

if (position == 0)

{ head = head.next

}

int count = 0; Node temp = head; Node temp1 = head

while (temp != null)

{

if (count == position)

{ temp1.next = temp.next

return head

}

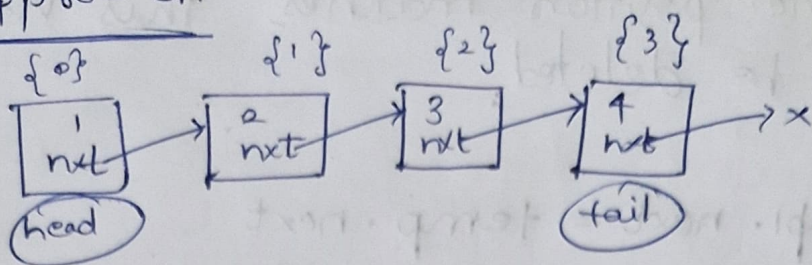
temp1 = temp // to keep track of previous element to

temp = temp.next

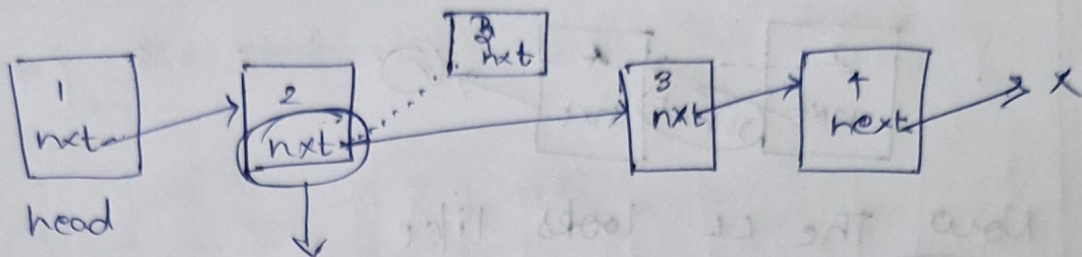
count++

join with next of delete Node

Approach

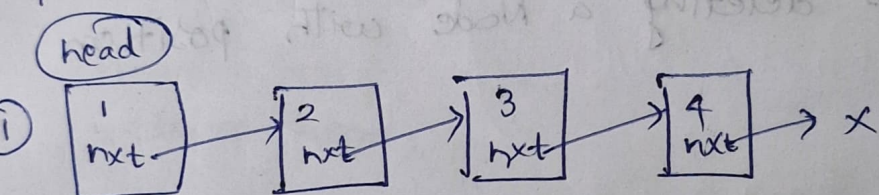


if want delete at position 2



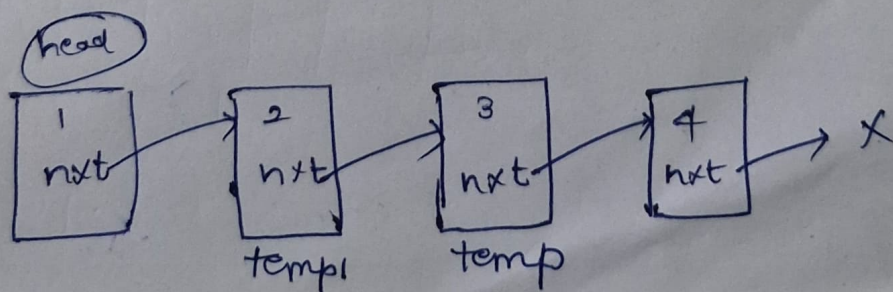
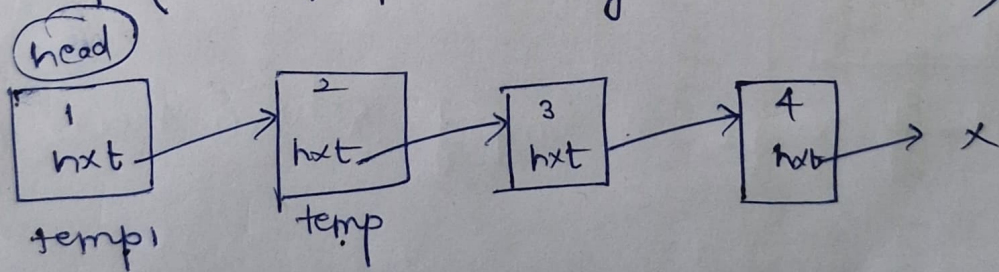
this should be connected to the node after the deletion of the node at position 2

→ So While moving temp forward, We should also keep track of previous temp.



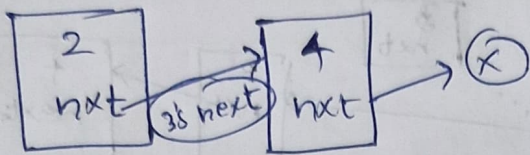
temp

temp1 (to keep track of previous temp)

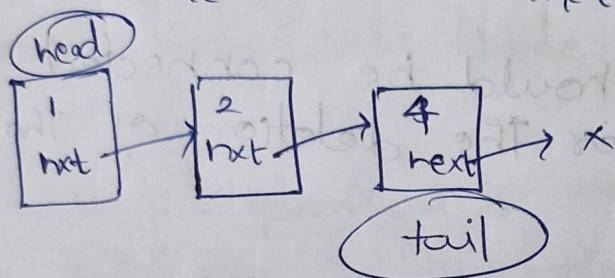


Now The position matches, this node has to be deleted,

So $\text{temp1.next} = \text{temp.next}$
(2's next) = (3's next) $\Rightarrow 4$

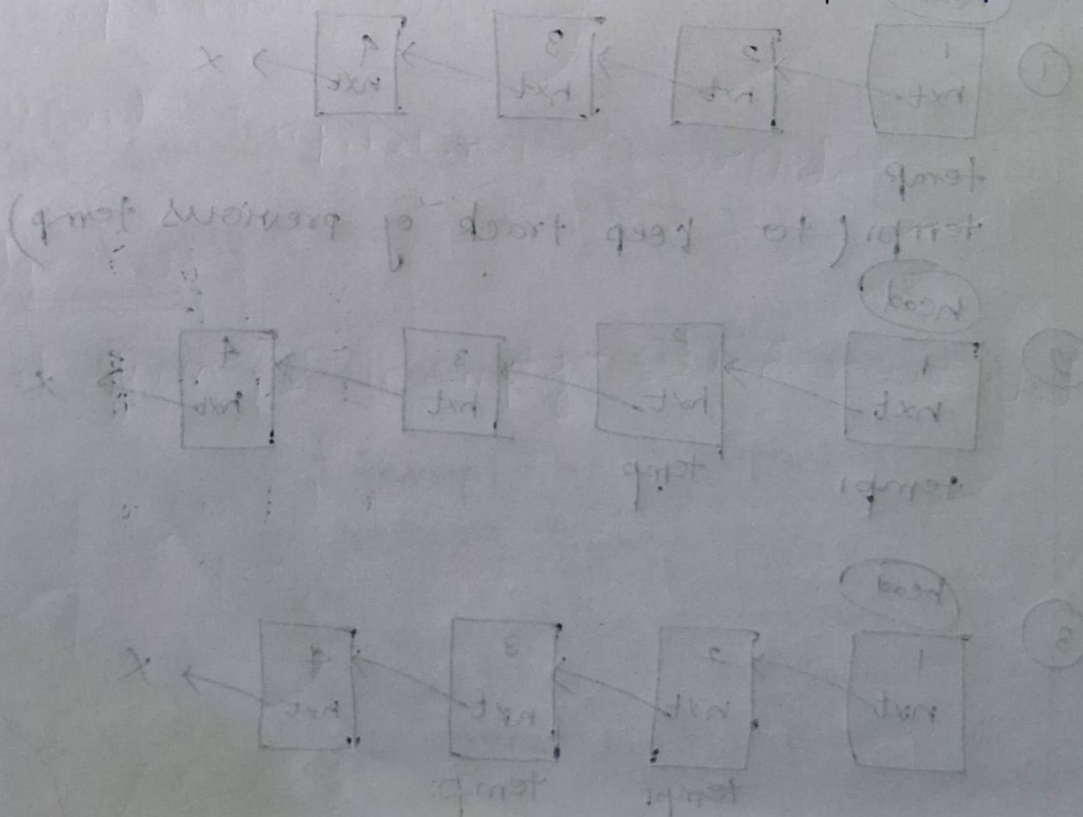


Now The LL looks like

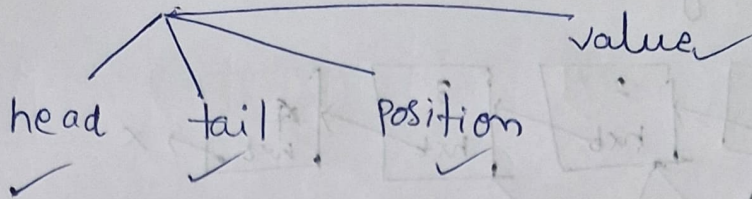


Delete The Node for the given value

Same as deleting a Node with position

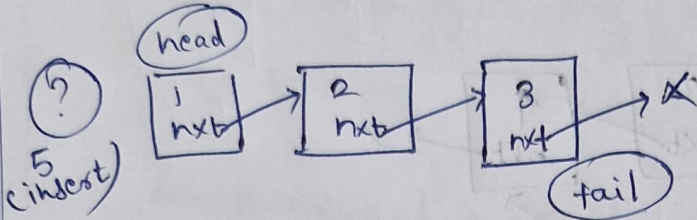


Insertion

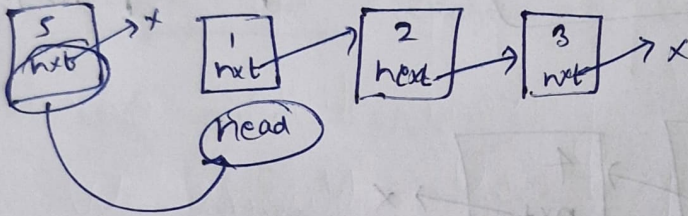
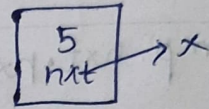


Insertion at head

Insert the element 5 at the head



Node temp = new Node(5);

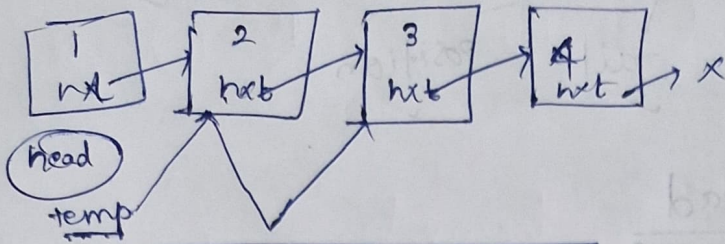
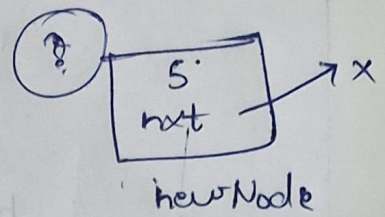


temp.next = head;
head = temp;
return head;

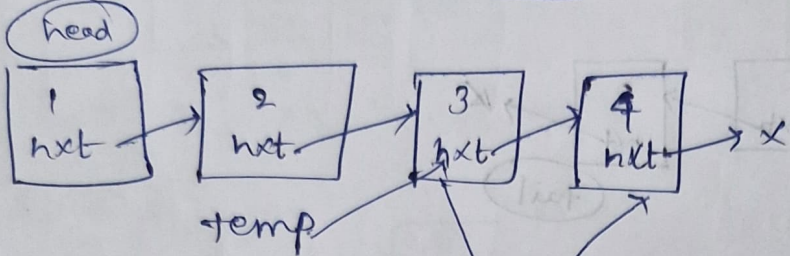
```

Node InsertAtHead(
private static Node head, int x)
{
    Node temp = new Node(x);
    temp.next = head;
    head = temp;
    return head;
}
    
```

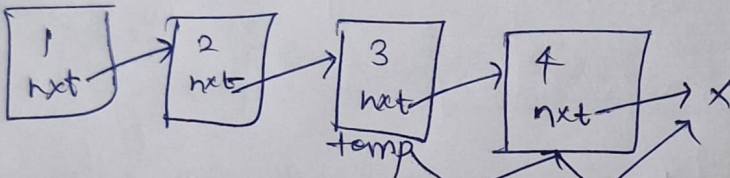
Inserting at last in LL



$\text{temp.next.next} = \text{NULL}$ $\times$ $\rightarrow \text{temp} = \text{temp.next}$



$\text{temp.next.next} = \text{NULL}$ $\times$ $\text{temp} = \text{temp.next}$



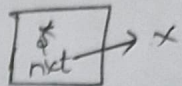
$\text{temp.next.next} = \text{NULL}$ $\times$

i) point $\text{temp.next.next} = \text{newNode}$

Edge Case

if given LL is empty return new Node

Code

```
private static Node insertAtEnd(Node head, int x)
{
    Node newNode = new Node(x) → 
    Node temp = head
    while (temp.next != NULL)
    {
        temp = temp.next
    }
    temp.next = newNode
```

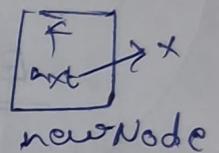
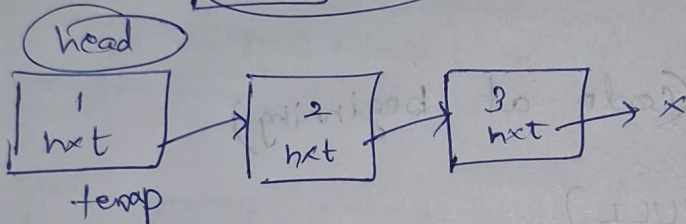
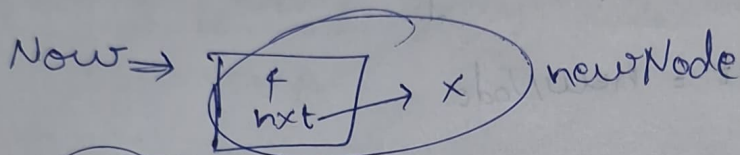
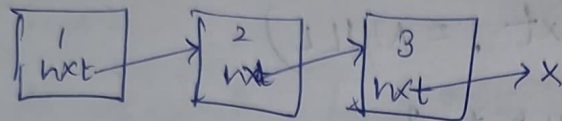
Edge Case (code at beginning)

```
if (head == NULL)
{
    return newNode;
}
if (head.next == NULL)
{
    head.next = newNode; return head;
}
return head;
}
```

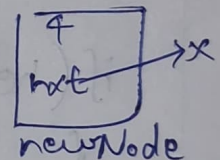
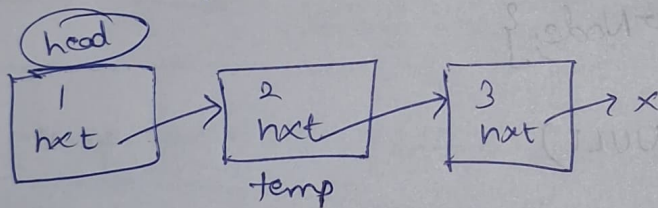
Inserting ~~Before~~ ^{after} kth element

$$K = (1 - N) \Rightarrow (1 - N + 1)$$

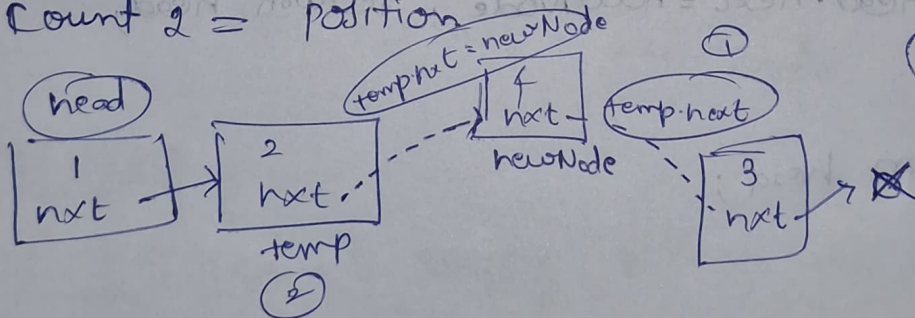
Inserting a element 2 position, data = 4



$\rightarrow$ Count 1 $\neq$ position, so move temp



$\rightarrow$ Count 2 = position



$\left\{ \begin{array}{l} \text{newNode.next} = \text{temp.next} \\ \text{temp.next} = \text{newNode} \end{array} \right.$

Code with Edge Case

```
private static Node insertAtK (Node head, int p, int d)
{
    // Edge Case
    Node newNode = Node(d);
    if (head == null)
    {
        if (p == 1)
        {
            return newNode;
        }
    }
    int count = 0;
    while (temp != null)
    {
        count++;
        if (count == p) // (count == p-1)
        {
            newNode.next = temp.next;
            temp.next = newNode;
            return head;
        }
        else
        {
            temp = temp.next;
        }
    }
    return head;
}
```

Inserting before the value

// same as position